

ЧЕК-ЛИСТ ФОРЕНЗИКИ cPanel/WHM

Ищем следы взлома после [CVE-2026-41940](#) · CVSS 9.8 · 64 дня не было оповещения от вендора

⚠ Если ваш сервер был доступен из интернета с 23 февраля по 28 апреля 2026 — считайте его скомпрометированным. Патч закрывает дыру, но не удаляет то, что уже занесли внутрь. Нужно проверить свой сервер. Ниже алгоритм проверки.

🔑 1. SSH-ключи — первое, что оставляют хакеры

Проверьте `authorized_keys` у `root` и каждого пользователя с доступом к серверу:

```
cat /root/.ssh/authorized_keys
cat ~/.ssh/authorized_keys

# Проверить у всех пользователей сразу
for u in $(cut -d: -f1 /etc/passwd); do
    f="/home/$u/.ssh/authorized_keys"
    [ -f "$f" ] && echo "=== $u ===" && cat "$f"
done
```

Любой ключ, которого вы не добавляли — немедленно удалить.

👤 2. Новые пользователи с UID 0

Хакеры создают технических пользователей с правами `root`. Также проверьте `authorized_keys` у `www-data`, `nobody` и других системных аккаунтов:

```
# Пользователи с UID 0 (только root должен иметь 0)
awk -F: '($3==0)' /etc/passwd

# Аккаунты, созданные недавно
awk -F: '{ print $1 }' /etc/passwd | tail -20
```

Если видите незнакомые имена с `UID 0` — это компрометация. Проверьте также `/etc/shadow` на новые записи.

🐞 3. Руткиты через `ld.so.preload`

Хакеры используют `/etc/ld.so.preload` для подгрузки библиотек, которые скрывают файлы и процессы на уровне ядра — обычные `ls` и `ps` их не покажут:

```
# Файл должен быть пустым или отсутствовать
cat /etc/ld.so.preload

# Проверить загруженные библиотеки процессов
ls -la /proc/*/exe 2>/dev/null | grep deleted
```

Команда `grep deleted` ищет процессы, которые удалили свой бинарник после запуска — классическая техника вредоносков. На нагруженном сервере будет шум — сверяйте имена.

👑 4. Подмена системных команд через алиасы и `PATH`

Хакеры переопределяют `ls`, `ps`, `top` — чтобы скрыть свои файлы и процессы:

```
alias
which ls ps top lsof
echo $PATH
type ls ps netstat
```

Пути должны вести в `/bin` или `/usr/bin`. Если видите `~/local/bin` или `/tmp` в начале `PATH` — тревога.

5. Скрытая автозагрузка (zsh/bash)

Проверьте файлы инициализации — туда вставляют бэкдор, который грузится при каждом логине:

```
cat ~/.zshrc ~/.zshenv
cat /etc/zsh/zshrc /etc/zsh/zshenv
cat /etc/profile /etc/profile.d/*.sh
cat /etc/bash.bashrc /root/.bashrc
```

Ищите: *eval*, *base64*, *curl*, *wget* — особенно в одну строку и с обфускацией.

6. Задачи планировщика (cron)

Патч не чистит crontab. Проверяйте под каждым пользователем:

```
crontab -l
sudo crontab -l
cat /etc/crontab
ls -la /etc/cron.d/ /etc/cron.hourly/ /etc/cron.daily/

# Crontab всех пользователей
for u in $(cut -d: -f1 /etc/passwd); do
    echo "=== $u ==="; crontab -u $u -l 2>/dev/null; done
```

7. Подозрительные systemd-сервисы

Хакеры регистрируют сервисы с правдоподобными именами вроде *cpanel-sync.service* для постоянной связи с командным центром:

```
systemctl list-units --type=service --state=running
systemctl list-units --type=service --state=enabled

# Сервисы, добавленные недавно
find /etc/systemd /lib/systemd -name "*.service" -mtime -75 2>/dev/null
```

Сверяйте незнакомые имена с документацией *cPanel*. Сервис без описания (*Description=*) — повод проверить подробнее.

8. Сетевая активность и процессы

Если *nuclear.x86* или *Sorry Ransomware* активны — они видны здесь:

```
sudo lsof -i -P -n | grep ESTABLISHED
ss -tulpn # замена устаревшего netstat
ps auxwwf
```

9. История команд

Злоумышленники часто чистят историю небрежно. Ищите следы загрузки полезной нагрузки:

```
grep -E "wget|curl|base64|python|perl|nc |ncat" ~/.zsh_history
grep -E "wget|curl|base64|python|perl|nc |ncat" ~/.bash_history
stat ~/.zsh_history ~/.bash_history
```

Если дата изменения истории совпадает с февралём–апрелем 2026 — основание для полной форензики.

10. Свежесозданные файлы и веб-шеллы

Поиск файлов, появившихся за период уязвимости, и PHP-бэкдоров в WordPress:

```
# Файлы, изменённые за последние 75 дней
find / -mtime -75 -type f -not -path "/proc/*" -not -path "/sys/*" 2>/dev/null

# Веб-шеллы в WordPress
find /home /var/www -name "*.php" -newer /etc/passwd 2>/dev/null
grep -r "eval(base64_decode" /home /var/www 2>/dev/null

# Бэкдоры в wp-content/uploads (там не должно быть PHP)
find /var/www -path "*/uploads/*.php" 2>/dev/null
```

11. Sorry Ransomware — проверка шифровальщика

Если шифровальщик отработал, файлы переименованы в .sorry. Проверяем, не начался ли процесс:

```
# Ищем зашифрованные файлы
find /home /var/www -name "*.sorry" 2>/dev/null

# Проверяем записки с требованием выкупа
find /home /var/www -name "HOW_TO_DECRYPT*" -o -name "README_SORRY*" 2>/dev/null
```

Если файлы .sorry найдены — сервер скомпрометирован. Не платите выкуп, изолируйте машину и восстанавливайте из резервной копии.

12. Ботнет nuclear.x86 — майнинг Монеро и сканирование сети

nuclear.x86 — вариант Mirai под x86. Жрёт CPU на Intel Xeon, майнит Монеро и сканирует сеть в поисках новых жертв. Прячет бинарник в /tmp или /dev/shm:

```
# Высокая нагрузка без понятного источника
top -b -nl | head -20

# Исходящие соединения на нестандартные порты (не 80/443/22)
ss -tnp | grep -v -E ":80|:443|:22"

# ELF-бинарники там, где им не место
find /tmp /var/tmp /dev/shm -type f -executable 2>/dev/null

# Процессы с нестандартными именами и высоким CPU
ps auxwwf | sort -rn -k3 | head -20
```

Если видите процесс с 90%+ CPU без понятного имени — это быстрее любых команд. Проверьте /proc/<PID>/exe на путь к бинарнику.

13. Когда утилитам нельзя доверять — смотрим мимо них

Нормальный руткит через ld.so.preload подменит top, ps, ss и lsof — они покажут чистую картину. Сначала проверьте сам preload:

```
# Если здесь что-то есть — все утилиты на этом сервере врут
cat /etc/ld.so.preload
```

Если файл не пустой — дальнейшая проверка с этого сервера бессмысленна. Переходите к rescue-режиму.

Если ld.so.preload пуст, но подозрения остались — читаем данные напрямую из ядра через /proc, минуя все утилиты:

```
# Загрузка CPU из ядра — top не участвует
cat /proc/stat

# Все процессы напрямую из procfs — ps не участвует
ls /proc | grep -E '^[0-9]+$' | while read pid; do
    cat /proc/$pid/cmdline 2>/dev/null | tr '\0' ' '
    echo " [PID $pid]"
done
```

```
# Сетевые соединения напрямую — ss и netstat не участвуют
cat /proc/net/tcp
cat /proc/net/tcp6
```

Единственный полностью надёжный способ — rescue-режим провайдера. Есть у Hetzner, DigitalOcean, OVH. Загружаетесь с чистого образа, монтируете диск и смотрите файловую систему снаружи:

```
# В rescue-режиме провайдера — монтируем диск
mount /dev/sda1 /mnt

# Проверяем preload с чистой системы
cat /mnt/etc/ld.so.preload

# Смотрим tmp и свежие библиотеки
ls -la /mnt/tmp /mnt/dev/shm
find /mnt -name "*.so" -mtime -75 2>/dev/null

# SSH-ключи с чистой системы
cat /mnt/root/.ssh/authorized_keys
```

В rescue-режиме руткит не загружен — всё что видите, реально. Это единственный способ проверить сервер, которому вы перестали доверять.

14. MySQL/MariaDB: конфигурационный предохранитель

Первым делом — проверить глобальную переменную:

```
SHOW VARIABLES LIKE 'secure_file_priv';
```

Три возможных результата: • NULL — норма: INTO OUTFILE полностью заблокирован • " (две одинарные кавычки, пустая строка) — критично: запись разрешена в любую директорию, петля самовосстановления активна • путь вида /var/lib/mysql — ограничено этим каталогом, но убедитесь что он не пересекается с web-контентом
Исправить пустое значение: добавьте в /etc/mysql/my.cnf → secure_file_priv = NULL

146. MySQL: init_connect и init_file

Через init_connect злоумышленник выполняет SQL при каждом подключении к базе — даже после удаления триггеров. init_file задаёт SQL-файл, который выполняется при старте сервера:

```
SHOW VARIABLES LIKE 'init_connect';
SHOW VARIABLES LIKE 'init_file';

# Если init_file указывает на файл — прочитайте его:
cat /путь/из/init_file
```

Любое непустое значение init_connect или init_file — немедленная проверка содержимого.

Сначала спросите сам сервер, где он держит плагины. Затем проверьте загруженные плагины — UDF можно загрузить напрямую через плагиновый механизм, минуя mysql.func:

```
-- Реальный путь к плагинам
SHOW VARIABLES LIKE 'plugin_dir';

-- Загруженные плагины с библиотеками (проверьте незнакомые)
SELECT PLUGIN_NAME, PLUGIN_STATUS, PLUGIN_LIBRARY
FROM information_schema.plugins
WHERE PLUGIN_LIBRARY IS NOT NULL;

-- UDF через CREATE FUNCTION
SELECT * FROM mysql.func;
```

Проверить файловую систему по найденному пути и в нестандартных местах:

```
# Свежие .so в plugin_dir (подставьте путь из SHOW VARIABLES)
find /usr/lib/mysql /usr/lib64/mysql -name "*.so" -mtime -75 2>/dev/null
```

```
# .so в /tmp и /dev/shm (там им не место)
find /tmp /var/tmp /dev/shm -name "*.so" 2>/dev/null

# Нестандартные .so, загруженные прямо сейчас
lsof | grep "\.so" | grep -vE "/(lib|lib64|usr/lib)/"
```

Незнакомая строка в mysql.funcs или PLUGINS, или свежий .so-файл с датой февраль–апрель — немедленная эскалация.

16. MySQL: триггеры, процедуры и EVENT SCHEDULER

У типовых CMS — WordPress, Bitrix, Joomla, OpenCart — триггеров в норме нет. Вредоносный триггер использует INTO OUTFILE, чтобы записать PHP-шелл обратно в wp-content/uploads/ сразу после того, как администратор его удалил. Это петля самовосстановления:

```
SELECT trigger_name, event_object_table, LEFT(action_statement, 100)
FROM information_schema.triggers
WHERE trigger_schema NOT IN ('mysql','sys','performance_schema');

SELECT routine_name, routine_type, LEFT(routine_definition, 100)
FROM information_schema.routines
WHERE routine_schema NOT IN ('sys','mysql','information_schema');

-- EVENT SCHEDULER: задачи, выполняющиеся по расписанию внутри БД
SHOW VARIABLES LIKE 'event_scheduler';
SELECT event_schema, event_name, status, LEFT(event_definition,80)
FROM information_schema.events
WHERE event_schema NOT IN ('mysql','sys');
```

Ищите в теле: base64, hex, eval, INTO OUTFILE, PREPARE/EXECUTE. Найденное — сохранить как артефакт, не удалять сразу.

PRO-совет: разорвать петлю самовосстановления на уровне ОС — проверьте, что папки веб-контента принадлежат веб-пользователю, а не mysql:

```
# Пользователь mysql не должен иметь write-доступ к web-контенту
ls -la /var/www/html | head -5
stat -c "%U %G %a" /home/*/public_html/wp-content/uploads 2>/dev/null
```

17. MySQL: опасные привилегии и фиктивные аккаунты

Хакеры создают аккаунты с правдоподобными именами — cpanel_backup, mysql_monitor, admin_support, service_account — не hacker123. Проверяйте по дате создания:

```
SELECT user, host, File_priv, Super_priv
FROM mysql.user
WHERE user = '' OR File_priv = 'Y' OR Super_priv = 'Y';

-- Пользователи по дате (MySQL 8.0+)
SELECT user, host, password_last_changed
FROM mysql.user ORDER BY password_last_changed DESC;
```

Если столбец password_last_changed недоступен (старый MySQL/MariaDB) — проверьте время изменения файлов в /var/lib/mysql/mysql/ за февраль–апрель 2026.

18. MySQL: бинарные логи — чёрный ящик за 64 дня

Если включена репликация или бэкапы cPanel, бинарные логи хранят полную историю действий за период уязвимости. Фильтруйте по дате, чтобы не гребать логи за несколько лет:

```
SHOW VARIABLES LIKE 'log_bin';
SHOW BINARY LOGS;

# Анализ с фильтрацией по периоду уязвимости
```

```
mysqlbinlog --start-datetime="2026-02-23 00:00:00" \  
--stop-datetime="2026-04-28 23:59:59" \  
/var/lib/mysql/mysql-bin.* \  
| grep -i "outfile\|create user\|grant\|drop"
```

В логах можно увидеть не только закладки, но и какие данные выкачал атакующий — например, экспорт таблиц с паролями пользователей.



19. Автоматизация: скрипт cpanel_forensics.sh

Все проверки из этого чек-листа — файловая система, SSH, cron, systemd, MySQL-блок — объединены в один bash-скрипт. Запускается с правами root, сохраняет отчёт в /tmp/:

```
sudo bash cpanel_forensics.sh  
  
# Отчёт сохраняется автоматически:  
# /tmp/forensics_report_ДАТА_ВРЕМЯ.txt
```

Скрипт проверяет: SSH authorized_keys у всех пользователей, UID 0, ld.so.preload, алиасы и PATH, автозагрузку, cron, systemd-юниты, сетевую активность, историю команд, веб-шеллы, файлы .sory — и полный блок MySQL включая secure_file_priv, plugin_dir, UDF, триггеры, привилегии и бинарные логи.



20. Что делать, если нашли следы

1. Изолируйте сервер от сети — отключите внешний доступ, оставьте только управляющий канал.
2. Сделайте снапшот диска до чистки — для форензики и возможных претензий к хостингу.
3. Безопасный дамп БД для восстановления (флаги обязательны — иначе перенесёте бэкдоры):
mysqldump --all-databases --skip-triggers --routines=FALSE --no-create-info > dump_clean.sql
4. Удалите чужие SSH-ключи, задачи cron, подозрительные сервисы, UDF-функции и .so-файлы.
5. Отзовите опасные привилегии: REVOKE FILE ON *.* FROM user@host — установите secure_file_priv=NULL в my.cnf.
6. WordPress: регенерируйте SALT-ключи в wp-config.php (<https://api.wordpress.org/secret-key/1.1/salt/>) — это сбросит все активные сессии, включая сессии хакера.
7. Смените BCE пароли: root, cPanel, FTP, БД, почта, API-ключи — с чистой машины.
8. Проверьте права на папки веб-контента: они должны принадлежать веб-пользователю, не mysql.
9. Если shared-хостинг — требуйте у провайдера письменный отчёт о проверке за февраль–апрель. Ответ «мы обновились» не считается.
10. Внедрите мониторинг аномалий (EDR/auditd/Wazuh). Не полагайтесь только на автообновления.

Батранков Денис · Канал по кибербезопасности: t.me/safebdv